TAREA M53 – CD-YAIR PALOMARES

```
#Importación de librerías
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings("ignore")

import os
os.chdir("C:/Users/yair_/OneDrive/Desktop/Ciencia de Datos/Módulo 52 - Análisis de regresión")
```

```
#Se lee la base de datos y se convierte en dataframe
data_house = pd.read_csv("kc_house_data.csv")
data_house
```

	id	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	...	grade	sqft_above	sqft_basement	yr_built	y
0	7129300520	20141013T000000	221900.0	3	1.00	1180	5650	1.0	0	0	...	7	1180	0	1955	
1	6414100192	20141209T000000	538000.0	3	2.25	2570	7242	2.0	0	0	...	7	2170	400	1951	
2	5631500400	20150225T000000	180000.0	2	1.00	770	10000	1.0	0	0	...	6	770	0	1933	
3	2487200875	20141209T000000	604000.0	4	3.00	1960	5000	1.0	0	0	...	7	1050	910	1965	
4	1954400510	20150218T000000	510000.0	3	2.00	1680	8080	1.0	0	0	...	8	1680	0	1987	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
21608	263000018	20140521T000000	360000.0	3	2.50	1530	1131	3.0	0	0	...	8	1530	0	2009	
21609	6600060120	20150223T000000	400000.0	4	2.50	2310	5813	2.0	0	0	...	8	2310	0	2014	
21610	1523300141	20140623T000000	402101.0	2	0.75	1020	1350	2.0	0	0	...	7	1020	0	2009	
21611	291310100	20150116T000000	400000.0	3	2.50	1600	2388	2.0	0	0	...	8	1600	0	2004	
21612	1523300157	20141015T000000	325000.0	2	0.75	1020	1076	2.0	0	0	...	7	1020	0	2008	

21613 rows × 21 columns

```python
#Limpieza de datos

#Se eliminan del dataframe las variables de id y zipcode. Ya que estas no son características directas con la casa
data_house = data_house.drop(columns=['id', 'zipcode'])

#Se toma el año de venta de la columna date, se convierte a int despues
data_house['date'] = data_house['date'].str[:4]
data_house['date'] = data_house['date'].astype(int)

#Se crea columna de antiguedad de la casa
data_house["yr_antique"] = data_house["date"] - data_house["yr_built"]

#Se eliminan las columnas de "date" y "yr_built"
data_house = data_house.drop(columns = ["date", "yr_built"])

#Agregando la variable de 1 para el intercepto
data_house.insert(0, 'Intercepto', 1)
```

```python
#Opción de formato numérico para evitar que se coloquen decimales con notación científica
np.set_printoptions(formatter={"float_kind":"{:f}".format})
```

```python
#Definir las matrices Xdata y Ydata
Xdata = data_house.drop(columns = ['price']).values
Ydata = data_house[["price"]].values

#Dividir en bases de entrenamiento y prueba
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(Xdata, Ydata, test_size = 0.3, random_state = 1)

x = x_train
y = y_train
```

```python
#Operaciones para obtener la matriz b
XT_X = np.matmul(np.matrix.transpose(x), x)
XT_X_inv = np.linalg.inv(XT_X)

XT_Y = np.matmul(np.matrix.transpose(x), y)
betas = np.matmul(XT_X_inv, XT_Y)

#Predicciones
Y_pred = x @ betas

#Residuales
Resid = y - Y_pred

#RSS
RSS = np.sum(Resid**2)

#TSS
TSS = np.sum((y - y.mean())**2)

#R2
RSq = 1 - RSS/TSS
print("Coeficiente de determinación R cuadrada =", RSq)

#R2 ajustada
n = x.shape[0]
p = x.shape[1]

RSqAj = 1 - (RSS/(n-p))/(TSS/(n-1))
print("Coeficiente de determinación R cuadrada ajustada =", RSqAj)
```

```
Coeficiente de determinación R cuadrada = -128.8903328008599
Coeficiente de determinación R cuadrada ajustada = -129.03646049972926
```

Debido a que algunas variables en esta base de datos tienen dependencia exacta, la matriz X_XT no tiene inversa, y los valores de los coeficientes salen enormes, lo que provoca que los residuales se disparen.

Se procede a eliminar las variables de "sqft_above" y "sqft_basement"

```python
#Se eliminan las columnas de "sqft_above" y "sqft_basement"
data_house = data_house.drop(columns = ["sqft_above", "sqft_basement"])
```

```python
#Definir las matrices Xdata y Ydata - DE NUEVO
Xdata = data_house.drop(columns = ['price']).values
Ydata = data_house[["price"]].values

#Dividir en bases de entrenamiento y prueba
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(Xdata, Ydata, test_size = 0.3, random_state = 1)

x = x_train
y = y_train
```

```python
#Operaciones para obtener la matriz b
XT_X = np.matmul(np.matrix.transpose(x), x)
XT_X_inv = np.linalg.inv(XT_X)

XT_Y = np.matmul(np.matrix.transpose(x), y)
betas = np.matmul(XT_X_inv, XT_Y)

#Predicciones
Y_pred = x @ betas

#Residuales
Resid = y - Y_pred

#RSS
RSS = np.sum(Resid**2)

#TSS
TSS = np.sum((y - y.mean())**2)

#R2
RSq = 1 - RSS/TSS
print("Coeficiente de determinación R cuadrada =", RSq)

#R2 ajustada
n = x.shape[0]
p = x.shape[1]

RSqAj = 1 - (RSS/(n-p))/(TSS/(n-1))
print("Coeficiente de determinación R cuadrada ajustada =", RSqAj)
```

```
Coeficiente de determinación R cuadrada = 0.6990322378980569
Coeficiente de determinación R cuadrada ajustada = 0.6987335204738838
```

Ciclo para stepwise con el criterio 1

Este programa va eliminando una por una las variables no significativas, eligiendo primero la de mayor valor p. Hasta que al final quedan puras variables significativas

```
import math
import scipy.stats

bandera = 0
while bandera == 0:
    #Definir las matrices Xdata y Ydata
    Xdata = data_house.drop(columns = ['price']).values
    Ydata = data_house[["price"]].values

    #Dividir en bases de entrenamiento y prueba
    from sklearn.model_selection import train_test_split
    x_train, x_test, y_train, y_test = train_test_split(Xdata, Ydata, test_size = 0.3, random_state = 1)

    x = x_train
    y = y_train

    #Operaciones para obtener la matriz b
    XT_X = np.matmul(np.matrix.transpose(x), x)
    XT_X_inv = np.linalg.inv(XT_X)

    XT_Y = np.matmul(np.matrix.transpose(x), y)
    betas = np.matmul(XT_X_inv, XT_Y)

    #Predicciones
    Y_pred = x @ betas

    #Residuales
    Resid = y - Y_pred

    #RSS
    RSS = np.sum(Resid**2)

    #TSS
    TSS = np.sum((y - y.mean())**2)

    #R2
    RSq = 1 - RSS/TSS
    print("Coeficiente de determinación R cuadrada =", RSq)
```

```python
#R2 ajustada
n = x.shape[0]
p = x.shape[1]

RSqAj = 1 - (RSS/(n-p))/(TSS/(n-1))
print("Coeficiente de determinación R cuadrada ajustada =", RSqAj)

#Cálculo de la varianza del error de regresión
s_cuad = RSS / (len(y) - x.shape[1])

#Cálculo de la desviación estándar del error de regresión
s = math.sqrt(s_cuad)

# Cálculo de las t's estadísticas para cada coeficiente de regresión
result_t = []
for i in range(x.shape[1]):
    t = float(betas[i] / (s * math.sqrt(XT_X_inv[i][i])))
    result_t.append(t)

grados_libertad = len(y) - x.shape[1]
#La t_critica se obtendrá a un nivel de confianza del 95% (Alfa = 5%)
t_critico = abs(scipy.stats.t.ppf(q = 0.025, df = grados_libertad))
print("Valor crítico de t", t_critico)

variables_no_significativas = []
for i in range(x.shape[1]):
    if (abs(result_t[i]) > t_critico):
        print("Beta", i, "es significativa") #Aquí se rechaza la hipótesis nula
    else:
        print("Beta", i, "NO es significativa") #Aquí NO se rechaza la hipótesis nula
        if i != 0:
            variables_no_significativas.append(i)

if len(variables_no_significativas) > 0:
    #Cálculo de valores p
    valores_p = []
    for i in range(len(variables_no_significativas)):
        p = scipy.stats.t.sf(abs(result_t[variables_no_significativas[i]]), df = grados_libertad) * 2
        print("Valor p de Beta", variables_no_significativas[i], ":", p)
        valores_p.apend(p)
    index_mayor_p = valores_p.index(max(valores_p))
    variables_independientes = data_house.drop(columns = ['price']).columns
    data_house = data_house.drop(columns = [variables_independientes[variables_no_significativas[index_mayor_p]]])
    print("Se elimina la variable", variables_independientes[variables_no_significativas[index_mayor_p]])
```

```python
    else:
        bandera = 1

    print("Las variables independientes del modelo son:", data_house.drop(columns = ['price']).columns)
    print("----------------------------------------")
```

```
Coeficiente de determinación R cuadrada = 0.6990322378980569
Coeficiente de determinación R cuadrada ajustada = 0.6987335204738838
Valor crítico de t 1.9601209658145293
Beta 0 es significativa
Beta 1 es significativa
Beta 2 es significativa
Beta 3 es significativa
Beta 4 es significativa
Beta 5 es significativa
Beta 6 es significativa
Beta 7 es significativa
Beta 8 es significativa
Beta 9 es significativa
Beta 10 es significativa
Beta 11 es significativa
Beta 12 es significativa
Beta 13 es significativa
Beta 14 es significativa
Beta 15 es significativa
Las variables independientes del modelo son: Index(['Intercepto', 'bedrooms', 'bathrooms', 'sqft_living', 'sqft_lot',
       'floors', 'waterfront', 'view', 'condition', 'grade', 'yr_renovated',
       'lat', 'long', 'sqft_living15', 'sqft_lot15', 'yr_antique'],
      dtype='str')
----------------------------------------
```

```python
#Comparación de resultados contra reporte automatizado
import statsmodels.api as sm
regressor = sm.OLS(y, x).fit()
print(regressor.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                      y   R-squared:                       0.699
Model:                            OLS   Adj. R-squared:                  0.699
Method:                 Least Squares   F-statistic:                     2340.
Date:                Mon, 16 Mar 2026   Prob (F-statistic):               0.00
Time:                        17:29:40   Log-Likelihood:            -2.0571e+05
No. Observations:               15129   AIC:                         4.114e+05
Df Residuals:                   15113   BIC:                         4.116e+05
Df Model:                          15
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const      -3.899e+07   1.72e+06    -22.730      0.000   -4.24e+07   -3.56e+07
x1         -3.078e+04   2153.082    -14.296      0.000    -3.5e+04   -2.66e+04
x2          3.486e+04   3708.089      9.401      0.000    2.76e+04    4.21e+04
x3           162.5679      3.904     41.645      0.000     154.916     170.220
x4             0.1070      0.052      2.047      0.041       0.005       0.209
x5          1.675e+04   3709.856      4.515      0.000    9476.831     2.4e+04
x6          5.745e+05   1.99e+04     28.839      0.000    5.35e+05    6.14e+05
x7          4.543e+04   2428.931     18.704      0.000    4.07e+04    5.02e+04
x8          2.938e+04   2691.641     10.915      0.000    2.41e+04    3.47e+04
x9          9.891e+04   2468.836     40.063      0.000    9.41e+04    1.04e+05
x10           23.0757      4.235      5.449      0.000      14.775      31.376
x11         5.639e+05    1.2e+04     47.088      0.000     5.4e+05    5.87e+05
x12        -9.286e+04   1.36e+04     -6.849      0.000   -1.19e+05   -6.63e+04
x13           28.7391      3.947      7.282      0.000      21.003      36.475
x14           -0.3207      0.082     -3.895      0.000      -0.482      -0.159
x15         2410.6941     83.219     28.968      0.000    2247.575    2573.813
==============================================================================
Omnibus:                    11019.450   Durbin-Watson:                   2.030
Prob(Omnibus):                  0.000   Jarque-Bera (JB):           531831.190
Skew:                           2.993   Prob(JB):                         0.00
Kurtosis:                      31.423   Cond. No.                     5.66e+07
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 5.66e+07. This might indicate that there are
strong multicollinearity or other numerical problems.
```

Ciclo para stepwise con el criterio 2

Este programa va eliminando una por una las variables no significativas, eligiendo primero la de mayor valor p. Hasta que al final quedan puras variables significativas

```
import math
import scipy.stats

bandera = 0
while bandera == 0:
    #Definir las matrices Xdata y Ydata
    Xdata = data_house.drop(columns = ['price']).values
    Ydata = data_house[["price"]].values

    #Dividir en bases de entrenamiento y prueba
    from sklearn.model_selection import train_test_split
    x_train, x_test, y_train, y_test = train_test_split(Xdata, Ydata, test_size = 0.3, random_state = 1)

    x = x_train
    y = y_train

    #Operaciones para obtener la matriz b
    XT_X = np.matmul(np.matrix.transpose(x), x)
    XT_X_inv = np.linalg.inv(XT_X)

    XT_Y = np.matmul(np.matrix.transpose(x), y)
    betas = np.matmul(XT_X_inv, XT_Y)

    #Predicciones
    Y_pred = x @ betas

    #Residuales
    Resid = y - Y_pred

    #RSS
    RSS = np.sum(Resid**2)

    #TSS
    TSS = np.sum((y - y.mean())**2)

    #R2
    RSq = 1 - RSS/TSS
    print("Coeficiente de determinación R cuadrada =", RSq)
```

```python
#R2 ajustada
n = x.shape[0]
p = x.shape[1]

RSqAj = 1 - (RSS/(n-p))/(TSS/(n-1))
print("Coeficiente de determinación R cuadrada ajustada =", RSqAj)

#Cálculo de la varianza del error de regresión
s_cuad = RSS / (len(y) - x.shape[1])

#Cálculo de la desviación estándar del error de regresión
s = math.sqrt(s_cuad)

# Cálculo de las t's estadísticas para cada coeficiente de regresión
result_t = []
for i in range(x.shape[1]):
    t = float(betas[i] / (s * math.sqrt(XT_X_inv[i][i])))
    result_t.append(t)

grados_libertad = len(y) - x.shape[1]

valores_p = []
valores_p_variables_no_significativas = []
variables_no_significativas = []
for i in range(x.shape[1]):
    aux_p = scipy.stats.t.sf(abs(result_t[i]), df = grados_libertad) * 2
    valores_p.append(aux_p)
    if aux_p <= 0.05:
        print("Beta", i, "es significativa, su valor p es:", aux_p)
    else:
        print("Beta", i, "NO es significativa, su valor p es:", aux_p)
        variables_no_significativas.append(i)
        valores_p_variables_no_significativas.append(aux_p)

if len(variables_no_significativas) > 0:
    index_mayor_p = valores_p_variables_no_significativas.index(max(valores_p_variables_no_significativas))
    variables_independientes = data_house.drop(columns = ['price']).columns
    data_house = data_house.drop(columns = [variables_independientes[variables_no_significativas[index_mayor_p]]])
    print("Se elimina la variable", variables_independientes[variables_no_significativas[index_mayor_p]])
else:
    bandera = 1

print("Las variables independientes del modelo son:", data_house.drop(columns = ['price']).columns)
print("----------------------------------------")
```

```
Coeficiente de determinación R cuadrada = 0.6990322378980569
Coeficiente de determinación R cuadrada ajustada = 0.6987335204738838
Beta 0 es significativa, su valor p es: 1.7140033262086808e-112
Beta 1 es significativa, su valor p es: 4.6353274359424006e-46
Beta 2 es significativa, su valor p es: 6.187848144746751e-21
Beta 3 es significativa, su valor p es: 0.0
Beta 4 es significativa, su valor p es: 0.04070344106947745
Beta 5 es significativa, su valor p es: 6.390907346524478e-06
Beta 6 es significativa, su valor p es: 4.431622528315255e-178
Beta 7 es significativa, su valor p es: 3.4036405514986187e-77
Beta 8 es significativa, su valor p es: 1.2377280657945402e-27
Beta 9 es significativa, su valor p es: 0.0
Beta 10 es significativa, su valor p es: 5.1348257952848224e-08
Beta 11 es significativa, su valor p es: 0.0
Beta 12 es significativa, su valor p es: 7.710006713274758e-12
Beta 13 es significativa, su valor p es: 3.454635926575706e-13
Beta 14 es significativa, su valor p es: 9.864033140107746e-05
Beta 15 es significativa, su valor p es: 1.2883411367832341e-179
Las variables independientes del modelo son: Index(['Intercepto', 'bedrooms', 'bathrooms', 'sqft_living', 'sqft_lot',
       'floors', 'waterfront', 'view', 'condition', 'grade', 'yr_renovated',
       'lat', 'long', 'sqft_living15', 'sqft_lot15', 'yr_antique'],
      dtype='str')
----------------------------------------
```

```python
#Comparación de resultados contra reporte automatizado
import statsmodels.api as sm
regressor = sm.OLS(y, x).fit()
print(regressor.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                      y   R-squared:                       0.699
Model:                            OLS   Adj. R-squared:                  0.699
Method:                 Least Squares   F-statistic:                     2340.
Date:                Mon, 16 Mar 2026   Prob (F-statistic):               0.00
Time:                        18:06:45   Log-Likelihood:            -2.0571e+05
No. Observations:               15129   AIC:                         4.114e+05
Df Residuals:                   15113   BIC:                         4.116e+05
Df Model:                          15
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const      -3.899e+07   1.72e+06    -22.730      0.000   -4.24e+07   -3.56e+07
x1         -3.078e+04   2153.082    -14.296      0.000    -3.5e+04   -2.66e+04
x2          3.486e+04   3708.089      9.401      0.000    2.76e+04    4.21e+04
x3           162.5679      3.904     41.645      0.000     154.916     170.220
x4             0.1070      0.052      2.047      0.041       0.005       0.209
x5          1.675e+04   3709.856      4.515      0.000    9476.831     2.4e+04
x6          5.745e+05   1.99e+04     28.839      0.000    5.35e+05    6.14e+05
x7          4.543e+04   2428.931     18.704      0.000    4.07e+04    5.02e+04
x8          2.938e+04   2691.641     10.915      0.000    2.41e+04    3.47e+04
x9          9.891e+04   2468.836     40.063      0.000    9.41e+04    1.04e+05
x10           23.0757      4.235      5.449      0.000      14.775      31.376
x11         5.639e+05    1.2e+04     47.088      0.000     5.4e+05    5.87e+05
x12        -9.286e+04   1.36e+04     -6.849      0.000   -1.19e+05   -6.63e+04
x13           28.7391      3.947      7.282      0.000      21.003      36.475
x14           -0.3207      0.082     -3.895      0.000      -0.482      -0.159
x15         2410.6941     83.219     28.968      0.000    2247.575    2573.813
==============================================================================
Omnibus:                    11019.450   Durbin-Watson:                   2.030
Prob(Omnibus):                  0.000   Jarque-Bera (JB):           531831.190
Skew:                           2.993   Prob(JB):                         0.00
Kurtosis:                      31.423   Cond. No.                     5.66e+07
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 5.66e+07. This might indicate that there are
strong multicollinearity or other numerical problems.
```

Ciclo para stepwise con el criterio 3

Este programa va eliminando una por una las variables no significativas, eligiendo primero la de mayor valor p. Hasta que al final quedan puras variables significativas

```
import math
import scipy.stats

bandera = 0
while bandera == 0:
    #Definir las matrices Xdata y Ydata
    Xdata = data_house.drop(columns = ['price']).values
    Ydata = data_house[["price"]].values

    #Dividir en bases de entrenamiento y prueba
    from sklearn.model_selection import train_test_split
    x_train, x_test, y_train, y_test = train_test_split(Xdata, Ydata, test_size = 0.3, random_state = 1)

    x = x_train
    y = y_train

    #Operaciones para obtener la matriz b
    XT_X = np.matmul(np.matrix.transpose(x), x)
    XT_X_inv = np.linalg.inv(XT_X)

    XT_Y = np.matmul(np.matrix.transpose(x), y)
    betas = np.matmul(XT_X_inv, XT_Y)

    #Predicciones
    Y_pred = x @ betas

    #Residuales
    Resid = y - Y_pred

    #RSS
    RSS = np.sum(Resid**2)

    #TSS
    TSS = np.sum((y - y.mean())**2)

    #R2
    RSq = 1 - RSS/TSS
    print("Coeficiente de determinación R cuadrada =", RSq)
```

```python
#R2 ajustada
n = x.shape[0]
p = x.shape[1]

RSqAj = 1 - (RSS/(n-p))/(TSS/(n-1))
print("Coeficiente de determinación R cuadrada ajustada =", RSqAj)

#Cálculo de la varianza del error de regresión
s_cuad = RSS / (len(y) - x.shape[1])

#Cálculo de la desviación estándar del error de regresión
s = math.sqrt(s_cuad)

# Cálculo de las t's estadísticas para cada coeficiente de regresión
result_t = []
for i in range(x.shape[1]):
    t = float(betas[i] / (s * math.sqrt(XT_X_inv[i][i])))
    result_t.append(t)

grados_libertad = len(y) - x.shape[1]

valores_p = []
valores_p_variables_no_significativas = []
variables_no_significativas = []
for i in range(x.shape[1]):
    aux_p = scipy.stats.t.sf(abs(result_t[i]), df = grados_libertad) * 2
    valores_p.append(aux_p)
    lci = float(betas[i]) - t_critico * s * math.sqrt(XT_X_inv[i][i])
    lcs = float(betas[i]) + t_critico * s * math.sqrt(XT_X_inv[i][i])
    if lci <= 0 and lcs >=0:
        print("Beta", i, "NO es significativa, porque su intervalo de confianza es de", lci, "a", lcs)
        variables_no_significativas.append(i)
        valores_p_variables_no_significativas.append(aux_p)
    else:
        print("Beta", i, "es significativa, porque su intervalo de confianza es de", lci, "a", lcs)

if len(variables_no_significativas) > 0:
    index_mayor_p = valores_p_variables_no_significativas.index(max(valores_p_variables_no_significativas))
    variables_independientes = data_house.drop(columns = ['price']).columns
    data_house = data_house.drop(columns = [variables_independientes[variables_no_significativas[index_mayor_p]]])
    print("Se elimina la variable", variables_independientes[variables_no_significativas[index_mayor_p]])
else:
    bandera = 1
```

```python
print("Las variables independientes del modelo son:", data_house.drop(columns = ['price']).columns)
print("----------------------------------------")
```

```
Coeficiente de determinación R cuadrada = 0.6990322378980569
Coeficiente de determinación R cuadrada ajustada = 0.6987335204738838
Beta 0 es significativa, porque su intervalo de confianza es de -42357481.5637699 a -35632143.47732202
Beta 1 es significativa, porque su intervalo de confianza es de -35000.49780974602 a -26559.89599387283
Beta 2 es significativa, porque su intervalo de confianza es de 27589.980664699382 a 42126.5847737649
Beta 3 es significativa, porque su intervalo de confianza es de 154.91612982917763 a 170.21959865821077
Beta 4 es significativa, porque su intervalo de confianza es de 0.004525927668606461 a 0.20942399722169427
Beta 5 es significativa, porque su intervalo de confianza es de 9476.831016003307 a 24020.36388865936
Beta 6 es significativa, porque su intervalo de confianza es de 535495.7358385847 a 613596.4633843898
Beta 7 es significativa, porque su intervalo de confianza es de 40670.02249522029 a 50192.01779321314
Beta 8 es significativa, porque su intervalo de confianza es de 24103.62095535923 a 34655.50405673306
Beta 9 es significativa, porque su intervalo de confianza es de 94070.97703464936 a 103749.41052430261
Beta 10 es significativa, porque su intervalo de confianza es de 14.775372662209548 a 31.375938598547663
Beta 11 es significativa, porque su intervalo de confianza es de 540409.2461609365 a 587354.1855664728
Beta 12 es significativa, porque su intervalo de confianza es de -119429.63648690873 a -66282.5084103734
Beta 13 es significativa, porque su intervalo de confianza es de 21.003127432607094 a 36.47498780153129
Beta 14 es significativa, porque su intervalo de confianza es de -0.4820574308344661 a -0.15929779240450864
Beta 15 es significativa, porque su intervalo de confianza es de 2247.575399815366 a 2573.8127758756464
Las variables independientes del modelo son: Index(['Intercepto', 'bedrooms', 'bathrooms', 'sqft_living', 'sqft_lot',
       'floors', 'waterfront', 'view', 'condition', 'grade', 'yr_renovated',
       'lat', 'long', 'sqft_living15', 'sqft_lot15', 'yr_antique'],
      dtype='str')
----------------------------------------
```